

PROJET TRAITEMENT DE DONNÉES

Élèves 1A

SPORTIX

Étudiant(e)s :
Bonavero Méline
Brugal Kévin
Dahan Gabriel

Encadrant :
Leroy Kévin

Table des matières

Introduction	2
1 Présentation du projet	3
1.1. Contexte	3
1.2. Besoins fonctionnels	3
1.3 Présentation des données	3
2 Conception de l'application	4
2.1. Modélisation	4
2.2. Architecture générale	5
2.3. Choix technologiques	7
3 Réalisation	9
3.1. Organisation du travail	9
3.2. Développement	9
Architecture et Validation du Système	9
Moteur de Commandes et Interface CLI	9
Pipeline de Traitement des Données	9
Persistance et Reproductibilité	10
3.3. Difficultés rencontrées	10
4 Tests et validation	11
4.1. Stratégie de test	11
4.2. Quelques résultats	11
4.3. Limites de l'application	11
Conclusion	12
Annexes	13
Annexe 1 : Retour personnel de Méline Bonavero	13
Annexe 2 : Retour personnel de Gabriel Dahan	13
Annexe 3 : Retour personnel de Kévin Brugal	13

Introduction

Le sport de haut niveau suscite un engouement croissant, particulièrement renforcé par le succès des Jeux Olympiques et Paralympiques de Paris 2024. Dans ce contexte, l'exploitation des données est devenue incontournable, qu'il s'agisse d'optimiser les performances des athlètes ou d'affiner les analyses tactiques. Cependant, l'hétérogénéité des disciplines sportives soulève un défi informatique majeur : comment concevoir un système capable d'unifier et de traiter efficacement cette diversité de données ?

C'est pour répondre à cette problématique que nous avons conçu **Sportix**, une application de gestion de résultats de compétitions développée dans le cadre du projet de traitement de données de première année à l'ENSAI. À partir de données brutes, Sportix permet de modéliser des entités complexes telles que des compétitions, des matchs, des équipes ou encore des joueurs et joueuses. L'application offre à chaque utilisateur la possibilité d'importer des données, de les traiter, d'établir des relations entre les différentes entités, puis de générer des statistiques personnalisées.

Ce rapport retrace l'intégralité de notre démarche à travers quatre axes majeurs. Nous débutons par une présentation du cadre du projet et des exigences fonctionnelles qui ont guidé le choix de nos données. La deuxième partie détaille la phase de conception, de la modélisation des entités à la définition d'une architecture logicielle modulaire inspirée des systèmes ORM. Nous exposons ensuite la réalisation technique, l'organisation de notre développement ainsi que les défis rencontrés lors de la programmation. Enfin, nous validons la robustesse de Sportix par l'analyse des tests effectués et l'évaluation critique de ses performances et limites.

1 | Présentation du projet

1.1. Contexte

Le cahier des charges fourni par le sujet nous imposait de concevoir une application Python respectant les bonnes pratiques de la programmation orientée objet. Un des principaux critères était notamment la généralisation du code afin que celui-ci puisse être réutilisé pour un autre projet, avec des objectifs ou des jeux de données différents.

Suite à de nombreux échecs pour d'autres projets ayant pour conséquence une perte fondamentale de budget, nous avons voulu sauver la mise en proposant une application modulable, simple d'utilisation, et correspondant à n'importe quel jeu de données sportives. Mais qu'est-ce qu'une bonne application de gestion de données sans quelques statistiques ? Nous avons alors décidé de permettre à l'utilisateur d'en générer : taux de victoire, leaderboard, derniers matchs joués, statistiques générales sur les jeux de données... tout y est !

1.2. Besoins fonctionnels

Notre application fonctionne donc de la manière suivante : l'utilisateur commence par se connecter à « Sportix ». S'il s'agit de sa première fois, il ne va pas avoir d'autres choix que de télécharger des données de sport. En se déconnectant plus tard, il pourra choisir de sauvegarder une configuration afin de recharger ses traitements de données automatiquement ensuite. Il peut charger ces données de manière massive (avec un fichier CSV ou plusieurs, concaténés), ou de manière manuelle, ligne par ligne (pour des tests, par exemple). Une fois les données chargées, des liens se font automatiquement suivant les identifiants rentrés (`team_id`, `match_id`, `player_id`, `match_id`, `competition_id`), il peut alors regarder des statistiques sur les différents objets créés. Par exemple, il peut consulter le classement d'une compétition, les gagnants d'un match, le taux de victoire d'une équipe, etc.

1.3 Présentation des données

Afin de mener à bien ce projet et donc de répondre à nos objectifs, nous avons traité différents jeux de données. Nous avons donc eu accès à 10 jeux de données sur 10 sports différents. Ceux-ci permettaient d'avoir un certain nombre d'informations relatives au sport associé. Par exemple, nous avons principalement le nom des joueurs ou le score du match. Néanmoins, l'un des enjeux majeurs était de trouver des similarités entre ces différentes données puisque, en fonction du sport sélectionné, certaines informations étaient manquantes, n'existaient pas ou étaient nommées différemment.

Nous avons donc pour projet de créer différentes classes relatives aux différents sports proposés dans notre application. Après discussions, nous en avons conclu qu'il était préférable que notre application fonctionne sur un maximum de jeux de données différents, indépendamment du sport choisi. C'est donc l'utilisateur qui, après s'être connecté, importe les données qu'il souhaite sur n'importe quel sport, et fait les liens requis.

Nous avons cependant choisi pour les entités créées de ne garder que les paramètres suivants :

- `_Player` : `first_name`, `last_name`, `birthdate`, `height`, `weight`, `country`, `player_id`, `team_id`.
- `_Team` : `name`, `name_abbreviation`, `team_id`, `competition_id`.
- `_Match` : `date`, `team_scores`, `duration`, `team_ids`, `match_id`; `competition_id`.
- `_Competition` : `name`, `competition_id`.

2 | Conception de l'application

2.1. Modélisation

L'application fonctionne en interface en ligne de commande (CLI), avec une console dédiée. Les commandes de l'application exécutées sont toutes précédées d'un préfix, qui est ; par défaut, mais qui peut être changé au sein de l'application. Ainsi, dans la suite nous ferons référence au préfix par ; sans perte de généralité. Exemple de commande : ;help.

La modélisation d'une entité se fait de la manière suivante :

1. Variables globales de classe

Trois variables de classe définissent une entité :

- **params** : Représente l'ensemble des paramètres qui sont passés à la classe (donc dans la méthode `__init__`), sous la forme d'un dictionnaire contenant en clé le nom du paramètre, et en valeur un dictionnaire contenant le type de la variable ainsi que son caractère requis ou non lors du traitement des données. Par défaut, nous avons supposé qu'aucun paramètre n'est requis dans les entités de cette application, pour une meilleure modularité des utilisations. Elle est **obligatoire** et doit contenir les mêmes valeurs que les paramètres de `__init__`.
- **relations** : Représente toutes les relations que peut avoir l'entité courante avec les autres entités de l'application. Cette variable prend la forme d'un dictionnaire contenant en clé le nom d'une variable de la classe courante, et en valeur le lien vers la variable correspondante d'une autre classe. L'unicité des valeurs jointes n'est pas vérifiée, mais toute valeur en double ne comptera que comme une et unique valeur (d'où la nécessité d'implémenter également la méthode spéciale `__eq__`).
- **methods** : Représente l'ensemble des méthodes de traitement de données qu'il est possible d'exécuter lorsque l'application est lancée, lors de l'utilisation de la commande ;analyze. Elle prend la forme d'un dictionnaire contenant en clé le nom de la méthode, et en valeur un dictionnaire contenant les paramètres ainsi que les liens qu'elles nécessitent pour fonctionner (par exemple, les équipes ont besoin de la relation avec les matchs joués pour donner le taux de victoire d'une équipe).

Note : La récupération des paramètres **params** aurait pu être faite de manière automatique dans le code de l'application avec le module `inspect` de python, mais nous avons décidé de n'utiliser que des modules pythons vus en cours dans ce projet, d'où la nécessité de les spécifier dans un dictionnaire.

2. Méthodes

Les méthodes nécessaires à la création d'une entité sont les suivantes :

- **__init__** : Les liens entre les paramètres et les attributs de classe y sont faits. Aucune vérification de type n'est faite à ce stade là, car c'est la variable globale **params** qui permet de gérer cela automatiquement au sein de l'application.
- **__repr__** : Cette méthode est cruciale pour afficher les différentes entités lorsqu'elles sont chargées dans l'application. On peut également définir **__str__**, qui n'est pas obligatoire car par défaut `__str__==__repr__` si **__str__** n'est pas définie.
- **__eq__** : Cette méthode est cruciale lorsqu'il s'agit de vérifier qu'une instance d'une entité est déjà ou non dans une liste (c'est le cas particulier d'utilisation dans l'application, lors du chargement des relations entre entités). Toutes les méthodes de traitement de données sur l'entité, à enregistrer ensuite dans **methods**. Elles ne sont pas nécessaires au sens strict, mais sont le cœur de tous les traitements de données de l'application.

Finalement, il ne faut pas oublier que chaque entité de l'application doit hériter de `AppEntity`, qui permet d'enregistrer les liens entre les différentes instances.

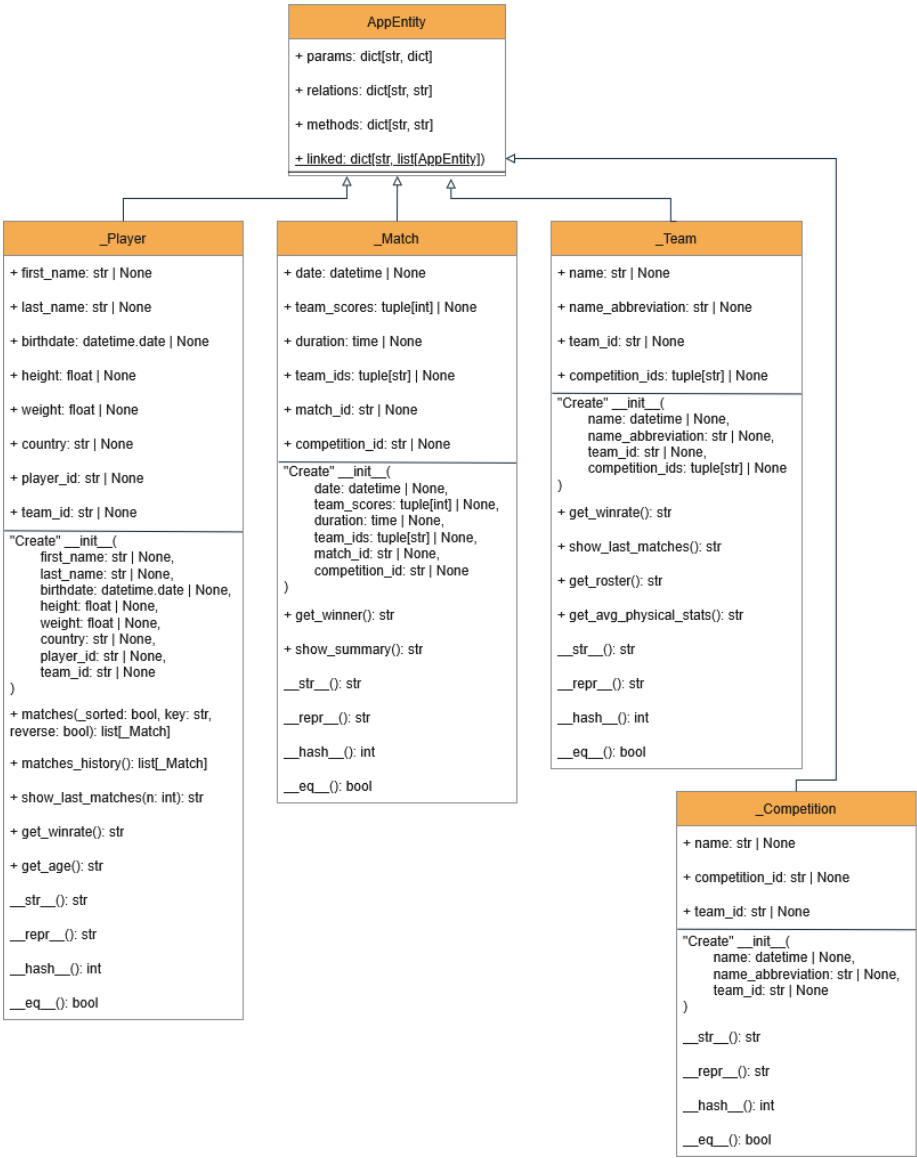
2.2. Architecture générale

Le projet suit une architecture classique de projet python :

```
sportix-project/
├── src/                                # Source code de l'application
│   ├── entities/                       # Entités
│   │   ├── base_entity.py             # Base class AppEntity
│   │   ├── competition.py
│   │   ├── match.py
│   │   ├── player.py
│   │   └── team.py
│   ├── application.py                 # Fichier principal de l'app.
│   ├── data_processing.py             # Traitement des données et relations
│   └── exceptions.py                  # Erreurs custom pour l'app.
├── tests/                              # Tests
│   ├── test_application.py
│   ├── test_competition.py
│   ├── test_data_processing.py
│   ├── test_entity.py
│   ├── test_match.py
│   ├── test_player.py
│   └── test_team.py
├── pyproject.toml                      # Config. du projet
└── README.md                           # Présentation et documentation
```

On trouve d'un côté la source `src/` avec les fichiers principaux tels que `application.py` ou `data_processing.py` ainsi que les entités `entities/`, et de l'autre les tests `tests/`.

Les entités définies dans le projet sont décrites dans le diagramme de classe ci-après, mais tout développeur peut les supprimer et en définir d'autre (promesse de modularité).



```
SportixApp

# POSSIBLE_ENTITIES: tuple[str]
# ENTITY_STR_TO_OBJECT: dict[str, AppEntity]
# AVAILABLE_COMMANDS: dict[str, Callable]
# config: dict[str, Any]
# data: dict[str, list(AppEntity)]
+ command_history: list[str]
+ prefix: str

"Create" __init__()

- clear_terminal()

- print_header()

- sportix_terminal_loop()

# analyze_entity_methods()

# get_entity_values(entity: str): list(AppEntity)

# ask_yn_question(q: str): bool

# ask_base_descriptors(choosen_entity: str, df: pd.DataFrame): pd.DataFrame

# ask_base_descriptors(choosen_entity: str, df: pd.DataFrame): pd.DataFrame

# handle_mass_upload(choosen_entity: str)

# handle_manual_upload(choosen_entity: str)

# handle_back_subcommand(id: int): int

# handle_process_subcommand(choosen_entity: str, df: pd.DataFrame): pd.DataFrame

+ load_data()

+ show_data()

+ analyze_data()

+ search_data()

+ change_command_prefix()

+ show_help()

+ output_config()

+ load_config()

+ exit_app()

+ run()

__str__(): str

__repr__(): str

__hash__(): int

__eq__(): bool
```

```
SportixAppDataProcessing

+ app: SportixApp

"Create" __init__(app: SportixApp)

- apply_processing_steps(df: pd.DataFrame, steps: list(dict)) -> pd.DataFrame

# represents_type(var: Any, type_: object): bool

# create_entities(choosen_entity: str, df: pd.DataFrame): list(AppEntity)

# link_data()

# get_dataframe(entity_name: str) -> pd.DataFrame

# cast_value(val: Any, target_type: type, inner_type: type): Any

+ get_general_statistics(entity_name: str): pd.DataFrame

+ get_value_counts(entity_name: str, param_name: str) -> pd.Series
```

2.3. Choix technologiques

- Architecture Logicielle :

Plutôt que d'utiliser une base de données relationnelle classique, nous avons fait le choix de développer un système de gestion d'entités inspiré des ORM, mais adapté à une structure en mémoire vive.

Modularité (AppEntity) : Toutes les entités (joueurs, équipes, matches) héritent d'une classe de base **AppEntity**. Ce choix permet de définir de manière déclarative les paramètres (**params**), les relations (**relations**) et les méthodes disponibles pour chaque entité.

Graphe de Relations Dynamique : Le système permet de lier automatiquement les objets entre eux (ex : lier un **_Player** à sa **_Team** via un identifiant) sans duplication de données, facilitant la navigation complexe entre les entités lors des analyses.

- Manipulation de Données : Pandas

Pour le cœur du traitement de données, nous avons intégré Pandas.

Performance : Pandas permet de manipuler des milliers de lignes de données (CSV) de manière très performante.

- Interface Utilisateur : CLI

L'application utilise une interface en ligne de commande.

Efficacité : Le terminal offre une rapidité d'exécution supérieure et permet de voir les informations plus simplement et dans l'ordre.

Extensibilité : L'ajout de nouvelles commandes se fait simplement par l'ajout de méthodes publiques dans la classe principale **SportixApp**, rendant le projet facilement évolutif.

- Qualité Logicielle et Environnement de Développement

Pour garantir la fiabilité du projet sur le long terme, nous avons mis en place une chaîne d'outils moderne :

Tests Automatisés (pytest) : Une suite de tests unitaires couvre les fonctionnalités critiques (chargement, calculs de statistiques, intégrité des relations).

Analyse Statique et Typage (mypy & ruff) : L'utilisation de **mypy** pour la vérification du typage et de **ruff** pour le linting assure une qualité de code constante et limite les erreurs à l'exécution.

Gestion des Dépendances (pyproject.toml) : L'utilisation d'un fichier **pyproject.toml** permet une gestion saine des versions des bibliothèques utilisées pour réinstaller le projet.

- Persistance et reproductibilité : un système de configuration

L'un des points forts de Sportix est sa capacité à sauvegarder et à restaurer l'intégralité d'une session de travail via un système de fichiers de configuration JSON. Contrairement à une simple sauvegarde de données, ce mécanisme enregistre toutes les étapes de la création des données.

Cette approche garantit une reproductibilité totale : un utilisateur peut interrompre son travail de nettoyage de données et le reprendre instantanément sur une autre machine, ou partager son fichier de configuration pour permettre à un collaborateur de re-générer les mêmes analyses à partir des sources brutes.

Résumé des technologies clés :

Domaine	Technologie	Justification
Traitement de données	Pandas	Puissance de calcul et traitement CSV
Structure	~ORM	Modularité et gestion des relations
Interface	CLI	Simplicité
Qualité	Pytest, Ruff, Mypy	Robustesse et maintenance
Reproductibilité	Configuration	Persistance sans sauvegarder de données

3 | Réalisation

3.1. Organisation du travail

Pour le projet, nous nous sommes organisés de la façon suivante :

- Gabriel : conception du coeur de l'application (input/output, data processing), des relations entre les entités avec `AppEntity` pour que le tout ressemble à un ORM ;
- Kévin : écriture des tests et vérifications, écriture des classes des entités ;
- Méline : écriture des classes des entités, écriture du rapport

A noter que cette répartition ne prend en compte que la proportion de travail que nous avons allouée à chaque partie concernée. Chacun a travaillé sur d'autres parties du projet, ainsi que sur ce rapport.

3.2. Développement

Le développement de l'application s'est articulé autour de la classe maîtresse `SportixApp`. Cette classe centralise la logique, la gestion de l'interface et la persistance de l'état de l'application.

Architecture et Validation du Système

Dès l'instanciation de `SportixApp`, un mécanisme de vérification de l'intégrité de la configuration est mis en place dans la méthode `__init__`, ce qui garantit que :

- Les entités disponibles (joueurs, équipes, matchs, compétitions) possèdent bien les propriétés requises (`params` et `relations`).
- La configuration est bien définie.
- Les commandes du terminal sont correctement liées à des méthodes exécutables sans arguments, évitant ainsi des erreurs lors de l'exécution.

Moteur de Commandes et Interface CLI

L'interface utilisateur repose sur un moteur de commandes extensible défini via la propriété `_AVAILABLE_COMMANDS`. Ce système permet de lier dynamiquement des mots-clés saisis par l'utilisateur (comme `analyze`, `load_data` ou `search`) à des fonctionnalités internes. La méthode `__sportix_terminal_loop` gère la lecture des entrées, l'application du préfixe de commande (par défaut ;) et le nettoyage automatique de l'écran pour maintenir une interface lisible.

Pipeline de Traitement des Données

Le développement du chargement des données a nécessité une logique pour transformer des fichiers CSV bruts en objets structurés :

1. **Mapping des descripteurs** : La méthode `_ask_base_descriptors` permet à l'utilisateur de lier interactivement les colonnes de son CSV aux paramètres attendus par les classes métiers.
2. **Transformations à la volée** : Via `_handle_process_subcommand`, nous avons implémenté des capacités de manipulation de données avancées directement en console. L'utilisateur peut ainsi combiner des colonnes, diviser des informations (`split`), réaliser des jointures entre fichiers ou créer des colonnes conditionnelles.
3. **Instanciation** : Une fois le traitement terminé, les entités sont instanciées grâce aux données et les potentielles relations avec d'autres entités déjà instanciées sont faites.

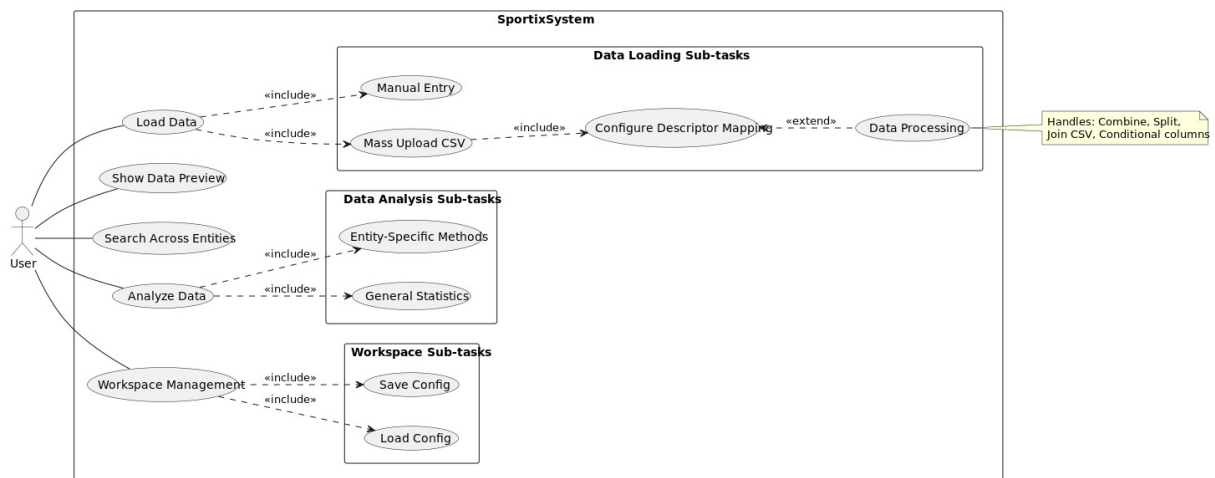
Persistance et Reproductibilité

Un aspect majeur du développement réside dans la gestion de la configuration via le format JSON. Plutôt que de sauvegarder les objets transformés (ce qui serait lourd et peu flexible), l'application enregistre le "chemin" de création des données dans `_config` :

- Les chemins des fichiers sources.
- Le mapping des colonnes choisi par l'utilisateur.
- La liste ordonnée des étapes de traitement appliquées.

La méthode `load_config` permet ainsi de recharger l'espace de travail en rejouant automatiquement toutes les étapes de traitement sur les sources brutes, garantissant une reproductibilité totale des analyses d'une session à l'autre.

Ce diagramme des cas d'utilisation résume les fonctionnalités finalement disponibles pour l'utilisateur :



3.3. Difficultés rencontrées

Notre première grande difficulté s'est trouvée être dès le début du projet. En effet, nous n'arrivions pas à faire les diagrammes UML car nous pensions qu'il fallait créer des classes prenant des spécificités de chaque sport, que nous n'avions pas encore. C'est d'ailleurs pour cette raison que nous n'avons pas gardé nos premiers diagrammes qui étaient trop spécifiques, ceux-ci étant trop éloignés de notre application finale. De plus, nous avons dû nous lancer dans le code, encore une fois, sans les jeux de données, d'où le fait, au départ, de vouloir créer des classes pour chaque sport.

Une fois les jeux de données disponibles, nous avons pris un certain temps à les trier et à trouver des similitudes entre les différents sports proposés. Ce travail n'ayant pas particulièrement abouti, nous avons décidé que chaque utilisateur pourrait télécharger ses propres données sur l'application, et si certaines ne sont pas connues, elles ne s'afficheront tout simplement pas. De la même manière, nous avons offert la possibilité à l'utilisateur de visualiser des statistiques, en fonction de leur disponibilité ou non.

4 | Tests et validation

4.1. Stratégie de test

La stratégie de test de Sportix repose sur une approche multi-niveaux visant à garantir la robustesse du moteur de traitement et la précision des analyses statistiques.

- Tests Unitaires (Pytest) : Chaque composant atomique de l'application est testé isolément. Nous avons créé des suites de tests pour les quatre entités principales (__Player, __Team, __Match, __Competition) afin de valider leurs méthodes de calcul (winrate, moyennes de scores, roster, etc.).
- Validation du Traitement de Données : Le module SportixAppDataProcessing fait l'objet de tests spécifiques pour valider les étapes de transformation (split, combine, join, conditional). Cela assure que les données brutes sont correctement nettoyées avant d'être injectées dans les objets métiers.
- Analyse Statique de Type (mypy src tests --strict) : Pour prévenir les erreurs d'exécution, l'intégralité du code source et des tests est soumise à une vérification de type stricte. Cela garantit qu'aucune variable None n'est manipulée par erreur et que les signatures de fonctions sont respectées.

4.2. Quelques résultats

Les campagnes de tests et de typage ont produit les résultats suivants :

- Couverture Fonctionnelle : 24 tests unitaires couvrent actuellement l'ensemble des fonctionnalités critiques. Tous les tests sont au statut "Pass", confirmant la fiabilité des calculs statistiques. Commande effectuée :

```
pytest tests --cov
```

Exemple : Validation du calcul de l'âge d'un joueur par rapport à la date du jour et du winrate dynamique (ex : 66.7% pour 2 victoires sur 3 matches).

- Fiabilité du Code : Résolution de 113 erreurs de typage statique. Le projet passe le check

```
mypy src tests --strict
```

4.3. Limites de l'application

Malgré sa robustesse actuelle, l'application présente certaines limites intrinsèques :

- Temps d'exécution mauvais pour des données massives : Puisque l'application traite les données en créant des objets et en les stockant ultimement dans des listes (de manière non persistante), le temps d'exécution de certaines commandes peut être long pour de très gros volumes de données, contrairement à de vrais ORMs qui optimisent ces étapes là, ou n'y passent même pas.
- Absence de Persistance : Sportix ne stocke aucune donnée. Toutes les données et configurations sont chargées en mémoire vive lors de l'exécution de l'application. Bien que l'export/import d'une configuration JSON soit disponible, cela nécessite d'effectuer à nouveau beaucoup de calculs en cas de chargement de la configuration.
- Interface CLI : Puisque l'interface est strictement textuelle, il est difficilement possible de montrer directement à l'utilisateur des courbes ou traitements graphiques. On aurait pu cependant mettre des résultats sous forme d'output de fichiers d'images.

Conclusion

En ouverture, ce projet a permis d'aller plus loin qu'une simple application de gestion sportive, puisque sa modularité permet désormais à n'importe quel développeur de reprendre le code et de changer les entités disponibles, d'en ajouter en suivant le format spécifique d'une entité Sportix détaillé en 2.1 Modélisation.

En effet, Sportix constitue désormais un petit ORM (Mapping objet-relationnel) interactif en interface de ligne de commandes.

Plus de détails sur l'installation globale de l'application, ainsi qu'un exemple d'implémentation minimale pour une nouvelle entité sont disponibles dans le fichier `README.md` du projet, sur GitHub.

Annexes

Annexe 1 : Retour personnel de Méline Bonavero

J'ai été, pour ce projet, légèrement mitigée. À première vue, je trouvais le sujet particulièrement intéressant et l'idée de créer notre propre application m'a beaucoup plu. Néanmoins, le fait de ne pas pouvoir accéder dès le départ aux jeux de données a vraiment été un frein puisque nous avançons dans le flou et que ce que nous avons fait s'est révélé plutôt inutile par la suite. Vis-à-vis du devoir maison d'IPOO, qui était sur un sujet plus explicite, j'ai donc moins eu l'impression de pouvoir comprendre les concepts et l'écriture du code dès le début. Je tiens tout de même à souligner que j'ai particulièrement aimé travailler en groupe avec Gabriel et Kévin, car ils m'ont énormément aidé sur de nombreux points de la programmation que je ne comprenais pas. Je suis donc très fière de notre application finale et reconnaissante d'avoir été dans ce groupe et de voir notre application fonctionnelle après tout ce travail.

Annexe 2 : Retour personnel de Gabriel Dahan

Ce projet a été pour moi une expérience particulièrement enrichissante, sur laquelle je me suis senti plutôt à l'aise. Il est vrai qu'au début du projet nous étions un peu perdus sur la manière de l'aborder. Nous avons en effet initialement une toute autre idée de ce qu'il fallait produire, ce qui nous a amené à recommencer deux fois le projet dans sa globalité.

Cependant, dès que les attendus ont été plus clairs, j'ai personnellement beaucoup aimé travailler sur ce projet. En effet, ça a été pour moi l'occasion idéale d'explorer en profondeur la notion d'ORM qui m'intéressait depuis quelque temps. En effet, bien que j'en ai déjà utilisé par le passé, je n'avais jamais eu l'occasion d'explorer en profondeur ce genre de systèmes passionnants. J'ai ainsi essayé de recréer un système permettant de faire des liens entre différentes entités sous forme de classes, à partir d'un jeu de données initiales quelconque.

Au-delà de l'aspect technique, la collaboration avec Méline et Kévin a été très positive. L'échange et le partage des connaissances ont permis de surmonter les difficultés initiales et d'aboutir à une application modulaire et fonctionnelle, "Sportix", dont je suis très satisfait.

Annexe 3 : Retour personnel de Kévin Brugal

Au début du projet j'ai partagé un sentiment partagé de beaucoup au sein de l'ENSAI, j'étais un peu perdu. En effet le commencement était assez difficile car les consignes étaient larges et j'avais peur qu'on fasse fausse route mais au fur et à mesure du déroulement du projet j'ai appris à l'apprécier. Gabriel était très expérimenté et Méline un peu moins que moi donc j'ai apprécié ce rôle de passerelle que j'avais de lier l'expertise de Gabriel et l'envie de faire et d'apprendre de Méline. J'ai donc à la fois appris des choses auprès de Gabriel et j'ai aussi pu développer ma pédagogie en vulgarisant certains concepts techniques. En plus de tout cela je me suis senti très bien avec mon groupe, on a fait de nombreuses sessions de travail sur un poste ou on s'assurait de tout avoir compris à chaque étape et j'ai aussi apprécié que les moments où il était impossible de travailler ensemble on s'est bien réparti les tâches comme faire les diagrammes UML, les tests, coder des nouvelles features ou même en imaginer des nouvelles et structurer le projet.